



An efficient privacy-preserving transformer inference scheme for cloud-based intelligent decision-making in AIoT[☆]

Mingshun Luo^a, Haolei He^a, Wenti Yang^b,*, Shuai Yuan^c, Zhitao Guan^a

^a School of Control and Computer Engineering, North China Electric Power University, Beijing, China

^b Pengcheng Laboratory, Shenzhen, China

^c Department of Finance, Operations, and Information Systems (FOIS), Brock University, Ontario, Canada

ARTICLE INFO

Keywords:

Artificial Intelligence of Things
Privacy-preserving machine learning
Private transformer inference
Secure multi-party computation
Homomorphic encryption

ABSTRACT

The Artificial Intelligence of Things (AIoT) is transforming modern society by combining the data-collection capabilities of IoT devices with the inference power of cloud-based large language models (LLMs). However, transmitting sensitive data to the cloud for intelligent decision-making raises significant privacy concerns. Cryptographic techniques such as homomorphic encryption (HE) and secure multi-party computation (MPC) provide promising solutions for privacy-preserving inference. However, existing schemes primarily target small-scale models and are inefficient when applied to Transformer-based LLMs, which involve large-scale matrix multiplications and complex non-linear functions, and deep model architectures. To address these challenges, we propose an efficient privacy-preserving Transformer inference scheme for cloud-based AIoT scenarios. Our framework integrates HE and MPC to ensure data confidentiality while minimizing computational and communication overhead. We design a fast HE-based matrix multiplication protocol using an offline-online collaborative pipeline and single instruction multiple data (SIMD)-based packing rules. Furthermore, we develop an accurate and efficient MPC-based non-linear function evaluation protocol using optimized piecewise polynomial approximation and integer-fraction decomposition. Experimental results show that our approach achieves 8.3×–91.6× faster in matrix multiplication, 1.4×–19× faster in non-linear function evaluation, and 3.5×–137.9× reduction in communication overhead with the LAN network, while maintaining lossless accuracy, thus enabling secure and scalable intelligent decision-making in AIoT environments.

1. Introduction

The proliferation of the Internet of Things (IoT) has connected billions of devices and created vast data ecosystems across various domains [1]. However, traditional IoT systems lack sufficient analytical and decision-making capabilities required to address complex, multimodal, and dynamic data environments [2,3]. The integration of Artificial Intelligence of Things (AIoT) thus emerged as a promising paradigm. By incorporating artificial intelligence (AI)'s advanced data comprehension and inference capabilities, AIoT enables intelligent and autonomous operations across diverse applications [4], including smart homes, industrial manufacturing, and urban infrastructure [5]. According to Gartner, AI-driven solutions are expected to be key drivers of IoT market growth, with the market projected to reach \$991 billion by 2028 [6].

AIoT leverages the capabilities of large language models (LLMs) in data understanding and inference to significantly enhance the intelligence and autonomy of IoT systems [7]. However, IoT systems often

involve multimodal data and complex analysis tasks that exceed the capabilities of lightweight models deployed at the edge [8]. In addition, edge devices typically have limited computational resources, making them inefficient for performing the large-scale matrix multiplications and complex non-linear functions required by Transformer-based model inference [9]. Consequently, AIoT systems perform lightweight data preprocessing at the edge and offload more resource-intensive processing tasks to the cloud. In scenarios involving stringent data privacy requirements, such as medical monitoring [10,11], intelligent surveillance [12], and industrial automation [13], transmitting raw user data to the cloud introduces potential privacy risks [14–16]. This problem limits the adoption of LLMs in privacy-sensitive AIoT applications [17–19].

Most existing privacy-preserving inference schemes target small-scale deep learning models, especially convolutional neural network (CNN) [20]. CryptFlow2 [21] introduces new two-party secure multi-party computation (MPC) protocols for secure comparison and division.

[☆] This article is part of a Special issue entitled: 'SPASS' published in Journal of Systems Architecture.

* Corresponding author.

E-mail address: yangwt@pcl.ac.cn (W. Yang).

Cheetah [22] builds upon homomorphic encryption (HE) to evaluate linear layers without expensive rotation operations and provides lightweight primitives for non-linear functions such as ReLU and truncation. Panther [23] further customizes HE arithmetic circuit for linear layers and optimizes the millionaire's protocol for non-linear activations. Moreover, schemes such as SIRNN [24], Diaa et al. [25], and AutoFHE [26] focus on privacy-preserving inference for neural networks using MPC or HE techniques.

However, Transformer-based LLMs introduce challenges that far exceed those in small scale CNNs [27–30]. In contrast to CNNs, Transformers require significantly larger matrix-matrix multiplications and incorporate more complex non-linear operations. The attention mechanism itself involves computing relationships among query, key, and value matrices, which demand multiple large-scale matrix multiplications [31]. Furthermore, advanced activation functions like GELU and normalization layers such as LayerNorm incur heavier computational loads than the simpler ReLU and BatchNorm used in CNNs [32]. When these computations are repeated across dozens or even hundreds of layers, the resulting overhead renders traditional privacy-preserving techniques for smaller models infeasible for Transformers. In recent years, efforts have begun to address privacy-preserving inference for Transformer architectures [31,33–35], primarily focusing on optimizing matrix multiplication and non-linear function evaluation:

(1) Schemes such as IRON [33], BOLT [34], and NEXUS [35] leverage the single instruction multiple data (SIMD) [36] capabilities of HE to pack matrices into SIMD ciphertexts. These ciphertexts are processed in parallel through homomorphic matrix multiplication. However, these designs typically rely on extensive ciphertext rotations, which are computationally expensive and represent a major performance bottleneck.

(2) Existing methods for evaluating non-linear functions typically fall into two categories: polynomial approximation and lookup tables (LUTs). For GELU approximation, schemes like PUMA [37], BumbleBee [31], and NEXUS [35] use polynomials of degree up to 4 or 6. In the case of the exponential function, PUMA [37], BumbleBee [31], NEXUS [35] and CipherGPT [38] rely on Taylor series expansions, with polynomial degrees as high as 64 to ensure accuracy. While LUTs serve as an alternative, they involve a trade-off between approximation error and memory or access overhead. These challenges make privacy-preserving inference on Transformers particularly difficult.

To address the above challenges, we propose a privacy-preserving transformer inference scheme that integrates HE and MPC, which mainly focuses on optimizing both matrix multiplications and non-linear function evaluations. (1) For HE-based matrix multiplications, to eliminate the high overhead of ciphertext rotations, we design an offline-online collaborative computation pipeline. In the offline phase, edge nodes split, replicate, and encrypt input matrices into a compact, parallel-friendly structure using carefully constructed SIMD packing rules. During the online phase, matrix multiplication is efficiently carried out by only performing parallel multiplication and accumulation over the corresponding SIMD ciphertexts, thereby significantly reducing computational cost. (2) For MPC-based non-linear function evaluations, we improve the approximation of the GELU activation by reformulating breakpoint selection in piecewise polynomial approximation as an optimization problem that minimizes the maximum error across intervals. This strategy not only lowers the approximation error but also reduces the polynomial degree. For Softmax, we introduce an input decomposition method that separates inputs into integer and fractional parts. This enables a minimal lookup table and a reduced approximation range, resulting in fast and highly accurate exponential function evaluation. The key contributions of this paper are as follows:

- We propose a privacy-preserving Transformer inference framework for intelligent decision making in AIoT. Data collected from end devices is processed at the edge and securely processed in the cloud through a private inference service.

- We design a private matrix multiplication protocol based on HE that leverages an offline-online collaborative computation pipeline. In the offline phase, matrices are replicated and encrypted into compact SIMD-packed ciphertexts on the edge. The online phase requires only multiplication and accumulation operations of the corresponding SIMD ciphertexts to achieve highly efficient matrix computation.
- We develop a private non-linear function evaluation protocol based on MPC. For GELU, we use an optimized piecewise polynomial approximation with breakpoint selection formulated as a minimization problem. For Softmax, we decompose inputs into integer and fractional parts, to enable a compact LUT and low-degree polynomial approximation.
- We conduct extensive evaluations of both individual building blocks and the end-to-end system. Compared to existing methods, our approach achieves 8.3×–91.6× faster in matrix multiplication, 1.4×–19× faster in non-linear function evaluation, 3.5×–137.9× reduction in communication overhead with the LAN network, and lossless accuracy comparable to plaintext inference.

This paper is organized as follows. Section 2 introduces preliminaries for our work. Section 3 describes our privacy-preserving Transformer inference framework for AIoT deployment. Sections 4 and 5 present our proposed private matrix multiplication protocol and non-linear function evaluation protocol. Our experimental evaluations are provided in Section 6. Lastly, related studies are introduced in Section 7 and this work is concluded in Section 8.

2. Preliminaries

2.1. Notations

Table 1 summarizes the notations frequently used in our scheme.

Table 1
Frequent notations.

Notation	Description
$\llbracket x \rrbracket_i$	Additive sharing of x held by P_i
$\llbracket x \rrbracket_i^B$	Boolean sharing of x held by P_i
$Enc()$	Encryption of plaintext
$E()$	Encoding of plaintext
$\boxplus$	Homomorphic addition
$\boxtimes$	Homomorphic multiplication
L	Multiplicative depth
$\tilde{a}$	Ciphertext
N'	Polynomial degree in CKKS
N	Number of SIMD slots, where $N = N'/2$

2.2. Transformer-based model

Most language models such as BERT [39], GPT [40], and Llama [41] are all built upon the Transformer architecture, which consists of an input embedding layer followed by multiple Transformer blocks. As shown in Fig. 1, each Transformer block contains an attention mechanism and a feed-forward network (FFN), each followed by a LayerNorm component. For the convenience of protocol design, we divide Transformer's computations into linear and non-linear operations.

Linear Operations. Linear operations primarily refer to matrix multiplications, which are classified into ciphertext–plaintext matrix multiplication (Ct-Pt MatMul) and ciphertext–ciphertext matrix multiplication (Ct-Ct MatMul).

Ct-Pt MatMul is used in the computation of query Q , key K , and value V of the attention mechanism, which are denoted as follows:

$$Q = A \cdot W_Q, K = A \cdot W_K, V = A \cdot W_V \quad (1)$$

where $A \in \mathbb{R}^{m \times n}$ denotes the input matrix and $W_Q, W_K, W_V \in \mathbb{R}^{n \times k}$ denotes the weight matrices for Q , K , and V , respectively. FFN is denoted as follows:

$$FFN(X) = GELU(X \cdot W_1 + b_1) \cdot W_2 + b_2 \quad (2)$$

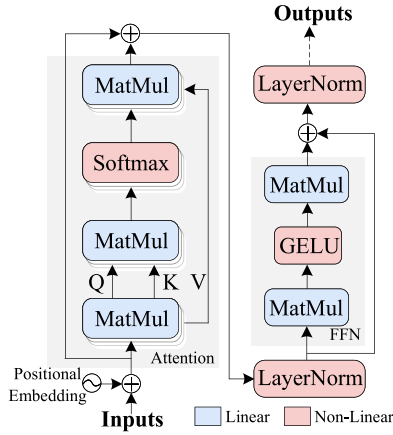


Fig. 1. Transformer-based model.

where W_1, W_2 are FFN weight matrices and b_1, b_2 are bias vectors.

Ct-Ct MatMul is used in two steps of the attention mechanism: multiplying Q by K^T and multiplying the resulting attention weight matrix S by V , where S is denoted as follows:

$$S = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \quad (3)$$

Non-Linear Operations. Non-linear operations mainly include GELU, Softmax, and LayerNorm. Specifically, GELU is denoted as follows:

$$\text{GELU} = 0.5x \cdot (1 + \text{erf}(x/\sqrt{2})) \quad (4)$$

where $\text{erf}(x) = (2/\sqrt{\pi}) \int_0^x e^{-t^2} dt$. Softmax is denoted as follows:

$$\text{Softmax}(x)[i] = \exp(x[i] - \bar{x}) / \sum_j \exp(x[j] - \bar{x}) \quad (5)$$

where $x[i]$ denotes row vector, $\bar{x}$ denotes the maximum value in $x[i]$. LayerNorm is denoted as follows:

$$\text{LayerNorm} = \gamma(x_i - \mu) / \sqrt{\sigma^2 + \epsilon} + \beta \quad (6)$$

where μ and σ are the mean and the standard deviation of the input, γ and β are learned parameters, and ϵ is a small constant to avoid division by zero.

Since LLMs contain many such layers, both linear and non-linear operations must be optimized for enabling privacy-preserving inference and reducing computational overhead.

2.3. Cryptographic primitives

We leverage two primary cryptographic primitives: homomorphic encryption (HE) and secure multi-party computation (MPC).

Homomorphic Encryption. HE allows computations over encrypted data [42]. The following $+$ operation:

$$\text{Dec}(sk, \text{Eval}(pk, +, c_1, c_2)) = \text{Dec}(sk, c_1) + \text{Dec}(sk, c_2) \quad (7)$$

and $\times$ operation can be performed without decrypting the data. HE includes Partially Homomorphic Encryption (PHE), Leveled Homomorphic Encryption (LHE), and Fully Homomorphic Encryption (FHE). The Key homomorphic Operations used is as below.

(1) SIMD Encryption $\text{Enc}()$ and Encoding $E()$: Pack a vector $\mathbf{a} = [a_0, a_1, \dots, a_{n-1}]$ into individual slots of a single homomorphic ciphertext $\tilde{a}$. This enables homomorphic operations to be performed in parallel across all slots.

(2) Addition ($\boxplus$) and Multiplication ($\boxtimes$): Perform element-wise addition and multiplication on SIMD ciphertexts $\tilde{a}_1, \tilde{a}_2$.

(3) Rescaling: After multiplications, apply $c' = \text{Rescale}(c)$ to reduce noise and maintain numeric precision.

(4) Rotation (SIMD Shift): Rotate the slots of a ciphertext by k positions through $c \langle k \rangle = \text{Rotate}(c, k)$. This operation incurs expensive automorphism and key-switching steps.

In this work, we adopt the Cheon–Kim–Kim–Song (CKKS) [43] leveled HE scheme, which enables floating-point computation while preserving precision. It supports computations up to a preset depth without expensive bootstrapping. In addition, it allows SIMD encoding by encrypting a vector $\mathbf{a} \in \mathbb{R}^N$ into a single ciphertext for parallel computing.

Secure Multi-Party Computation. MPC allows multiple parties to jointly implement a computation task while keeping their respective data private [44,45]. It enables n parties $P_1, P_2, \dots, P_n$, each holding a private input x_i , to jointly evaluate a public function $f(x_1, x_2, \dots, x_n) = (y_1, y_2, \dots, y_n)$ without revealing individual input, where y_i are outputs for P_i . MPC can be implemented using secret sharing (SS), oblivious transfer (OT), and other cryptographic protocols.

(1) Secret Sharing (SS): In a (t, n) scheme, a secret S is divided into n shares such that any subset of fewer than t shares yield no information about S , while any t or more shares can reconstruct the original secret precisely. This threshold property ensures both confidentiality against colluding adversaries and the correctness of reconstruction.

(2) Oblivious Transfer (OT): In the standard 1-out-of- k OT (denoted as k -OT), the sender possesses a message vector $(m_0, m_1, \dots, m_{k-1})$, and the receiver has a choice index $b \in 0, \dots, k-1$ to obtain message m_b . The protocol guarantees that the receiver learns only the selected messages, whereas the sender learns nothing about the receiver's choice, thereby preserving privacy for both sides.

In this work, we adopt a 2-out-of-2 additive secret-sharing MPC, which is computationally lightweight for linear operations and well suited to our two-party inference architecture. For a secret $x \in \mathbb{Z}_{2^l}$, parties P_1 and P_2 hold shares $\llbracket x \rrbracket_i$ such that $x = \llbracket x \rrbracket_1 + \llbracket x \rrbracket_2 \pmod{2^l}$, where l denotes the bit-length of the secret.

3. System description

3.1. Overview

In AIoT applications such as smart homes, smart cities, and industrial automation, IoT devices collect users' sensitive data (e.g. facial images, voice recordings, motion patterns) through various sensors [7]. While these devices use AI for intelligent support, lightweight edge-deployed models are insufficient for handling multimodal data and complex analysis tasks. In addition, they are limited in computational capacity and precision required for high-quality inference. As a result, inference tasks are typically offloaded to the cloud, which may lead to potential data leakage during transmission. Thus, we propose a privacy-preserving inference framework for AIoT systems, which integrates HE and MPC to ensure user data and model privacy while minimizing computational and communication overhead. An overview of the framework is shown in Fig. 2.

- **End Device:** The end device collects environmental data such as motion, location, images, and audio using sensors such as inertial measurement units, wireless modules, cameras, microphones. It then transmits this data to a nearby edge node for analysis.
- **Edge Node:** With limited data processing and computational capabilities, the edge node processes incoming data from the end device and forwards it to a cloud server for inference. Due to the sensitive nature of the data, this transmission is handled through privacy-preserving inference protocols.
- **Cloud Server:** The cloud server hosts large Transformer-based models with sufficient computational resources. It performs encrypted-domain inference using multiple privacy-preserving protocols (e.g., HE and MPC) to ensure data privacy and model protection.

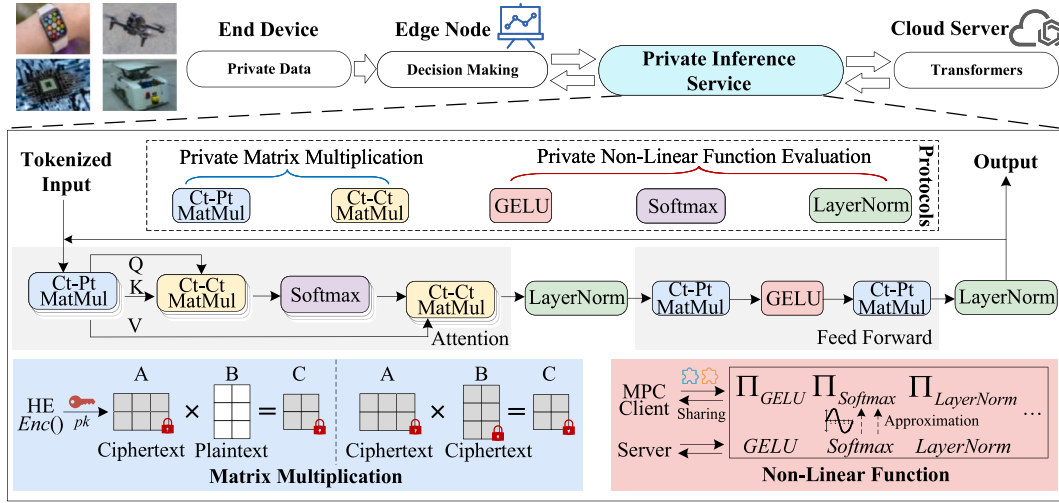


Fig. 2. Privacy-preserving transformer inference framework for AIoT.

To address intensive matrix multiplications and non-linear function evaluations in Transformer models, we analyze the computational characteristics of matrix multiplication and nonlinear function evaluation in Transformers, and compare the strengths of HE and MPC for these operations. Matrix multiplication involves a large number of multiply-and-accumulate operations. When implemented with MPC, it requires frequent communication, which leads to substantial communication overhead. In contrast, HE only requires a single communication round and can process multiple elements in parallel within one operation with the support of SIMD-style packing, significantly reducing communication costs during matrix multiplication. On the other hand, nonlinear function evaluation remains challenging under HE, since nonlinear operations cannot be directly supported. Even when using polynomial approximations, the high approximation degree typically results in heavy computation overhead. However, MPC naturally supports nonlinear protocols such as comparison and division, making it more efficient for these operations.

Therefore, our framework uses a HE-MPC hybrid protocol. HE, specifically, the leveled CKKS scheme, is used for efficient matrix multiplication. The edge node encrypts input matrices, and the cloud server performs computation directly on ciphertexts without decryption. MPC is employed for secure evaluation of non-linear functions using a 2-out-of-2 additive secret-sharing scheme between the edge node and the cloud server. Based on this design, we propose two optimized protocols for privacy-preserving Transformer inference that both ensure high inference efficiency and minimize the risk of data leakage:

Private matrix multiplication protocol: enables both Ct-Pt MatMul and Ct-Ct MatMul based on HE.

Private non-linear function evaluation protocol: performs secure evaluation of GELU, Softmax, and LayerNorm using two-party computation.

The workflow for a single Transformer layer in private inference proceeds as follows: ① Tokenized input and attention queries enter the attention module, where a Ct-Pt MatMul computes the encrypted-plaintext products to generate the Q, K, and V matrices. ② Ct-Ct MatMul multiplies Q with the transpose of K. ③ Private Softmax protocol securely evaluate the Softmax function. ④ Ct-Ct MatMul multiplies the Softmax output with V to produce the attention scores. ⑤ Private LayerNorm protocol normalizes the attention scores. ⑥ In the FFN, a Ct-Pt MatMul is followed by the private GELU protocol, and then another Ct-Pt MatMul is applied. ⑦ A second private LayerNorm protocol is carried out. ⑧ The output is passed to the next Transformer layer, or returned as the model output if it is the last one.

3.2. Threat model

We follow a two-party client-server model, where the client (edge node) C holds sensitive input data x and the server (cloud server) S hosts pre-trained model parameter w . To protect both parties' privacy, C refuses to reveal its sensitive input data x , and S refuses to disclose its pre-trained model parameter w . The MPC function is defined as follows:

$$f(data, model) = (decision, \emptyset) \quad (8)$$

where decision is the inference result returned to C and the $\emptyset$ signifies that S receives no output.

We assume a semi-honest threat model against a static, probabilistic polynomial time (PPT) adversary within the real world and the ideal world paradigm [46]. Specifically, we assume the existence of a simulator S that can emulate an ideal world for a given protocol Π , which corresponds to an ideal functionality $\mathcal{F}$. In this ideal world, the parties send their data to $\mathcal{F}$, which faithfully executes the computation and returns the outputs. Meanwhile, in the real world, the protocol Π is executed by the cloud server, according to the specification, even in the presence of an adversary $\mathcal{A}$. Security is guaranteed if, for any adversary $\mathcal{A}$, the view and output distributions in the real world are computationally indistinguishable from those in the ideal world:

$$\{View_{\mathcal{A}}^{\Pi}(\kappa, \langle x \rangle), output^{\Pi}(y)\} \stackrel{c}{\approx} Sim(\kappa, \langle x \rangle, \mathcal{F}(\langle x \rangle_1, \langle x \rangle_2, \dots, \langle x \rangle_n)) \quad (9)$$

where κ is the secure parameter. $View_{\mathcal{A}}^{\Pi}$ is the view of $\mathcal{A}$ in the execution of Π on x , $output^{\Pi}$ is the output of all parties.

In this model, both C and S strictly follow the protocol but may attempt to infer private information by analyzing intermediate messages exchanged during the interactions.

3.3. Private transformer inference and design goals

Our system allows edge nodes to transmit encrypted data to the cloud and receive inference results securely, resulting in intelligent, privacy-preserving decision support. The cloud performs the proposed protocols for secure Transformer inference using a hybrid of HE and MPC. The design achieves the following key goals:

- **Privacy-preserving:** The system ensures that neither the edge node's data nor the cloud server's model parameters are disclosed during inference. This guarantees end-to-end confidentiality for both parties.
- **Efficiency:** By optimizing the use of HE and MPC, the system minimizes computational latency and communication overhead, achieving a practical trade-off between security and performance.

- **Lossless Accuracy:** The proposed protocols are carefully designed to maintain the same level of inference accuracy as plaintext execution, thereby providing support for real-world AIoT applications where both accuracy and privacy are critical.

4. Private matrix multiplication protocol

4.1. Overview

Transformer inference operations are generally categorized into linear and non-linear computations. This section focuses on the Private Matrix Multiplication Protocol, which is an HE-based protocol for linear operations. Our protocol achieves efficient matrix multiplication through compact SIMD-based packing and parallel computation.

To eliminate costly rotation operations, we introduce an offline-online collaborative computation pipeline. In the offline phase, the edge node splits, replicates, and encrypts the matrix into a compact, parallel-friendly format based on carefully designed SIMD packing rules. In the online phase, matrix multiplication is performed efficiently through straightforward multiplication and accumulation of the corresponding SIMD ciphertexts, thereby significantly reducing computational overhead.

Since HE supports computations on ciphertexts and provides SIMD parallelism [43], we design two types of matrix multiplication: Ct-Pt MatMul and Ct-Ct MatMul. Ct-Pt MatMul occurs primarily in the attention mechanism when computing Q , K , V , as well as twice in the FFN. Ct-Ct MatMul is used when multiplying Q by K^T , and applying the attention weights matrix S to V , where S is defined as Eq. (3).

Our implementation assumes matrix dimensions $m = 128$, $n = 768$, and $k = 64$ as in BERT-base and GPT-2. We set the CKKS polynomial degree $N' = 16384$, which produces $N = N'/2 = 8192$ SIMD slots.

4.2. Ct-Pt MatMul

To accelerate privacy-preserving Transformer inference, we design an offline-online collaborative computation pipeline and a compact SIMD-based packing scheme that leverages the parallelism computing provided by HE. A key challenge in HE-based matrix multiplication is the excessive number of costly rotation operations required when matrices are naively packed. Although strategies such as the Baby-step Giant-step algorithm reduce this burden [34], their overhead remains substantial. To address this, we leverage the computational capacity of the edge node to perform matrix preprocessing during an offline phase, thereby enabling efficient SIMD-parallel computation during the online phase on the cloud. This design eliminates rotation operations and minimizes computation overhead. Furthermore, since the Transformer model is pre-trained and its weights remain constant, we preprocess the weight matrix W on the cloud in advance to enable efficient ciphertext–plaintext matrix multiplications at inference time.

Suppose the edge node tokenizes a user's input, e.g., for data-analysis or intelligent-decision making, producing an input matrix $A \in \mathbb{R}^{m \times n}$ and a weight matrix $W \in \mathbb{R}^{n \times k}$. Here, A contains the edge node's private information and must be encrypted before transmission to the cloud server. The model weights W , which are private to the cloud server, remains unencrypted as computation takes place on the cloud server only. To illustrate our approach, we present a toy example in Fig. 3, where $A \in \mathbb{R}^{2 \times 3}$ and $W \in \mathbb{R}^{3 \times 2}$.

Let $a_{ij} \in \mathbb{R}$, $b_{ij} \in \mathbb{R}$, $c_{ij} \in \mathbb{R}$ denote the (i, j) th element of matrices A , W , and C , respectively. c_{ij} is denoted as follows:

$$c_{ij} = \sum_{s=1}^n a_{is} b_{sj}, \forall i \in [1, m], j \in [1, k] \quad (10)$$

During offline computation phase, edge node splits the input matrix $A \in \mathbb{R}^{m \times n}$ into n column vectors $\mathbf{a}_i$. It then replicates the m elements of each $\mathbf{a}_i$ k times to create n new vectors $\mathbf{a}_i'$ for SIMD parallel computation. Each $\mathbf{a}_i'$ is homomorphically encrypted by encrypting matrix

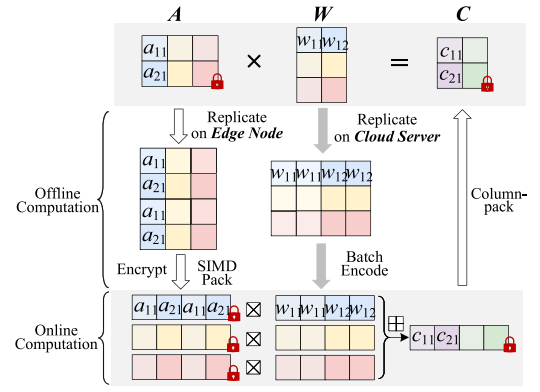


Fig. 3. A toy example of Ct-Pt MatMul.

column-by-column and packing them into SIMD slots. Then the following n SIMD ciphertexts $\tilde{a}_i$ are derived and sent to the cloud server:

$$\tilde{a}_i = \text{Enc}(\underbrace{\{a_{1i}, a_{2i}, \dots, a_{mi}, a_{1i}, a_{2i}, \dots, a_{mi}, \dots, a_{1i}, a_{2i}, \dots, a_{mi}\}}_{m \cdot k}) \quad (11)$$

where $i \in [1, n]$.

On the other hand, the cloud server splits the weight matrix $W \in \mathbb{R}^{n \times k}$ into n row vectors $\mathbf{w}_i$. It then replicates each of the k elements in $\mathbf{w}_i$ m times to create n new vectors $\mathbf{w}_i'$ for parallel computation as well. Since privacy-preserving inference is performed entirely on the cloud server, W does not require encryption. Instead, each row of $\mathbf{w}_i'$ is encoded into SIMD slots using HE's batch encoding method. This produces n SIMD plaintext encodings b_i as follows for parallel ciphertext–plaintext matrix multiplication:

$$b_i = E(\underbrace{[b_{i1}, b_{i1}, \dots, b_{i1}, b_{i2}, b_{i2}, \dots, b_{i2}, \dots, b_{ik}, b_{ik}, \dots, b_{ik}]}_{m \cdot k}) \quad (12)$$

where $i \in [1, n]$.

In our scheme, the matrix dimensions $m = 128$, $n = 768$, and $k = 64$ are consistent with those used in BERT-base and GPT-2, and each HE ciphertext contains $N = N'/2 = 8192$ SIMD slots. The edge node generates n SIMD ciphertexts $\tilde{a}_i$ and the cloud server prepares n SIMD plaintext encodings b_i . Each of these vectors packs exactly $m \times k$ elements, thereby fully occupying the available slots and achieving a compact packing that maximizes computational efficiency. Note that the slots of the column-packed SIMD ciphertexts $\tilde{a}_i$ and the row-packed SIMD plaintext encodings b_i are one-to-one aligned. By multiplying the i th $\tilde{a}_i$ with b_i and then summing over all n results, we derive a single SIMD ciphertext representing the column-major packed result matrix $C = A \cdot W \in \mathbb{R}^{m \times k}$.

Therefore, during the online computation phase, the cloud server performs HE multiplications between each pair of the n SIMD ciphertexts $\tilde{a}_i$ and their corresponding SIMD plaintext encodings b_i , followed by a summation of n results to obtain matrix C . The detailed algorithm is presented in Algorithm 1.

4.3. Ct-Ct MatMul

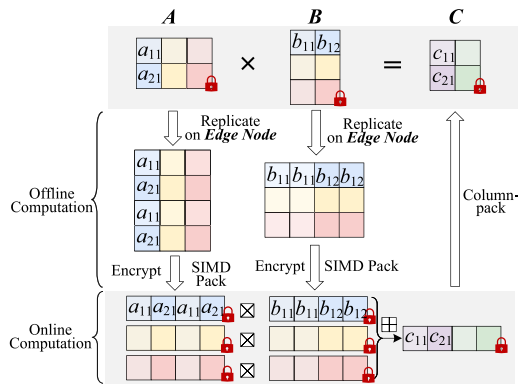
Ciphertext–ciphertext matrix multiplication primarily arises in the attention mechanism when multiplying Q by K^T and multiplying S by V , where S is defined as Eq. (3).

The method for Ct-Ct MatMul follows a structure similar to that of Ct-Pt MatMul. Given two matrices A and B , the objective is to compute $C = A \times B$ in the encrypted domain. Both A and B are preprocessed on the edge node during the offline phase, and then encrypted and sent to

```

1 Input: matrix  $A \in \mathbb{R}^{m \times n}$ , matrix  $W \in \mathbb{R}^{n \times k}$ 
2 Output: matrix  $C \in \mathbb{R}^{m \times k}$ 
3 Function Off_Comp_Edge( $A$ ):
4     // Offline phase (edge node)
5     for  $i = 1$  to  $n$  do
6          $a_i = A[:, i] \in \mathbb{R}^m$ 
7          $a'_i = [a_i, a_i, \dots, a_i] \in \mathbb{R}^{m \cdot k}$ 
8              $\underbrace{\hspace{1.5cm}}_k$ 
9          $\tilde{a}_i = \text{Encrypt\_SIMD}(a'_i)$ 
10    end
11    return  $\tilde{a}_i$ 
12 End
13 Function Off_Comp_Cloud( $W$ ):
14     // Offline phase (cloud server)
15     for  $i = 1$  to  $n$  do
16          $w_i = W[i, :] \in \mathbb{R}^k$ 
17          $w'_i = [b_{i,1}, \dots, b_{i,1}, b_{i,2}, \dots, b_{i,2}, \dots, b_{i,k}, \dots, b_{i,k}] \in \mathbb{R}^{k \cdot m}$ 
18              $\underbrace{\hspace{1.5cm}}_m \quad \underbrace{\hspace{1.5cm}}_m \quad \underbrace{\hspace{1.5cm}}_m$ 
19          $b_i = \text{Encode\_SIMD}(w'_i)$ 
20    end
21    return  $b_i$ 
22 End
23 Function On_comp( $\tilde{a}_i, b_i$ ):
24     // Online phase
25     for  $i = 1$  to  $n$  do
26          $\tilde{c}_i = \text{HE.Mul}(\tilde{a}_i, b_i)$ 
27          $\tilde{C} = \text{HE.Add}(\tilde{C}, \tilde{c}_i)$ 
28    end
29    return  $C$ 
30 End

```



the cloud server for computation. To illustrate our approach, we present a toy example with $A \in \mathbb{R}^{2 \times 3}$ and $B \in \mathbb{R}^{3 \times 2}$ in Fig. 4.

Although Ct-Pt and Ct-Ct matrix multiplication follow a similar pipeline, both consisting of offline preprocessing and online homomorphic SIMD-based computation, their essential difference lies in the party responsible for processing the right matrix and the timing of that processing. In Ct-Pt, the right matrix is processed by the cloud server, and the preprocessing can be completed before inference begins. In

In both $Q \times K^T$ and $S \times V$, the number of ciphertext slots is fixed, and the total number of elements across the packed matrices aligns with an integer multiple of those slots. This design enables the use of full-slot packing, which maximizes memory and computational efficiency. The final result C is also represented in column-major format using packed ciphertexts.

This section introduces our protocol for non-linear operations: the Private Non-Linear Function Evaluation Protocol. Particularly, the Private GELU Protocol leverages piecewise low-degree polynomial approximation with optimized breakpoints. The Private Softmax Protocol approximates the exponential function using integer-fraction separation. And the Private LayerNorm Protocol addresses reciprocal square root optimization via Newton’s iteration.

GELU is a widely used activation function in Transformer models. However, its non-linear nature makes it incompatible with secure computation schemes without approximation. Most existing approaches approximate GELU using high-degree polynomials [31,34], but these incur significant computational overhead and barely optimize the placement of breakpoints in piecewise approximations. We address both issues by introducing a piecewise low-degree polynomial approximation with optimized breakpoints.

Let the GELU function be denoted as Eq. (4). Hence:

where $P(x) = 0.5x \cdot \text{erf}(x/\sqrt{2})$

We observe that for sufficiently small x , GELU approximates to 0; for large x , GELU approximates to x . Since σ is even, we only need to approximate over $x \geq 0$. Accordingly, we define the approximation interval as $[0, b]$, where $b > 0$. This interval is partitioned into the following subintervals: On $[0, a]$, where $0 < a < b$, we approximate $P(x)$ using a cubic polynomial; On $[a, b]$, we use a second cubic polynomial. Also, on $[b, \infty)$, $P(x) = 0.5x$. To generate these approximations, we apply Chebyshev method to fit $P(x)$.

$$\min_{a,b} \max_{x \in \mathbb{R}} \{ \sup_{x \in [0,a]} |Error(x)|, \sup_{x \in [a,b]} |Error(x)|, \sup_{x \in [b,\infty)} |Error(x)| \} \quad (14)$$

We then derive the optimal solution: $a = 1.64$ and $b = 4$. Thus, the complete approximation $ApproxGELU(x)$ is presented as Eq. (15).

6

Algorithm 2: Piecewise Polynomial Approximation with Optimized Breakpoints

Input: approximate range $[0, 5]$, tolerance error $\delta \in \mathbb{R}^+$, origin function $P(x)$, Chebyshev nodes number n , polynomial degree d

Output: Breakpoints α and β , max error $error_max$, the parameter set of the piecewise polynomial over $[0, \alpha]$ $coef f_0$, parameter set over $[\alpha, \beta]$ $coef f_1$

```

1 Function Poly_Approx():
2   Initialize  $\alpha \leftarrow \alpha_0, \beta \leftarrow \beta_0$ 
3   for  $k = 1$  to  $n$  do
4     // Generate Chebyshev nodes
5      $nodes0[k] \leftarrow 0.5\alpha + 0.5\alpha \cdot \cos((2k-1)\pi/2n)$ 
6      $nodes1[k] \leftarrow 0.5(\alpha + \beta) + 0.5(\beta - \alpha) \cdot \cos((2k-1)\pi/2n)$ 
7   end
8   for  $i = 0$  to  $2$  do
9     if  $i < 2$  then
10      // Fit degree- $d$  polynomials and compute fit error
11       $coef f_i \leftarrow \text{polyfit}(nodes_i, P(nodes_i), d)$ 
12       $fit\_error_i \leftarrow \max |\text{poly}(coef f_i, x\_test_i) - P(x\_test_i)|$ 
13    else
14       $fit\_error_2 \leftarrow \max |0.5 \cdot x\_test_2 - P(x\_test_2)|$ 
15    end
16  end
17   $error\_max(\alpha, \beta) \leftarrow \max(fit\_error_i)$ 
18   $bounds \leftarrow [(0.1, 3), (3, 4.9)]$ 
19   $result\_global \leftarrow \text{differential\_evolution}(error\_max, bounds, \delta)$ 
20  // Global search
21   $result\_local \leftarrow \text{minimize}(result\_global, error\_max, bounds)$  // Local optimize
22  return  $\alpha, \beta, error\_max, coef f_i$ 
23 End

```

$$\begin{cases}
x, x \geq 4 \\
F_0(x) = 0.01115482|x|^3 - 0.11943534|x|^2 + 0.92723666|x| - 0.51157709 + 0.5x, 1.64 < |x| \leq 4 \\
F_1(x) = -0.112892447|x|^3 + 0.467559071|x|^2 - 0.0141870370|x| + 0.000466652113 + 0.5x, |x| \leq 1.64 \\
0, x \leq -4
\end{cases} \quad (16)$$

In our Private GELU Protocol, we first compute $|x|$ through the Most Significant Bit (MSB) protocol Π_{MSB} and select the appropriate polynomial segment by the less-than protocol Π_{LT} . We then apply Horner's rule to minimize the number of multiplications. Finally, we adopt the boolean-arithmetic multiplication protocol Π_{MulBA} to obtain the output of $ApproxGELU(x)$. The entire private GELU protocol is shown in Algorithm 3.

5.2. Private softmax protocol

The primary computational bottleneck in Softmax lies in evaluating the exponential function, which is both non-linear and resource-intensive under secure computation. Existing approaches, such as IRON [33], use LUTs, while others like Bumblebee [31] are based on direct polynomial approximations by the Taylor series:

$$\exp(x) \approx (1 + x/2^r)^{2^r} \quad (17)$$

However, these methods often find difficult to balance accuracy and efficiency, either incurring significant overhead or compromising numerical precision. Thus, we propose an efficient exponential function approximation method based on integer-fraction separation. This approach combines a small LUT with a cubic polynomial approximation to enable fast and accurate computation of $\exp(x)$ in privacy-preserving settings.

Algorithm 3: Private GELU Protocol

Input: edge node holds secret share $\llbracket x \rrbracket_A$, cloud server holds secret share $\llbracket x \rrbracket_B$, function $y = ApproxGELU(x)$

Output: secret share $\llbracket y \rrbracket_A$ and $\llbracket y \rrbracket_B$

```

1 Function Private_GELU():
2   edge node and cloud server jointly compute  $|x| = MSB(x)? - x : x$ 
3   and get secret share  $\llbracket |x| \rrbracket_A, \llbracket |x| \rrbracket_B$ 
4   jointly compute:  $\llbracket b_0 \rrbracket^B = \Pi_{LT}(\llbracket |x| \rrbracket, 1.64)$  //  $b_0 = 1\{|x| < 1.64\}$ 
5    $\llbracket b_1 \rrbracket^B = \Pi_{LT}(\llbracket |x| \rrbracket, 4)$  //  $b_1 = 1\{|x| < 4\}$ 
6    $\llbracket b_2 \rrbracket^B = \Pi_{LT}(4, \llbracket |x| \rrbracket)$  //  $b_2 = 1\{4 < x\}$ 
7   compute  $\llbracket z_0 \rrbracket^B = \llbracket b_0 \rrbracket^B$  //  $z_0 = 1\{|x| < 1.64\}$ 
8    $\llbracket z_1 \rrbracket^B = \llbracket b_0 \rrbracket^B \oplus \llbracket b_1 \rrbracket^B$  //  $z_1 = 1\{1.64 < |x| < 4\}$ 
9    $\llbracket z_2 \rrbracket^B = \llbracket b_2 \rrbracket^B$  //  $z_2 = 1\{x > 4\}$ 
10  and compute  $\llbracket F_0(x) \rrbracket$  and  $\llbracket F_1(x) \rrbracket$ 
11  compute  $\llbracket y \rrbracket = \Pi_{MulBA}(\llbracket z_0 \rrbracket^B, \llbracket F_0(x) \rrbracket) + \Pi_{MulBA}(\llbracket z_1 \rrbracket^B, \llbracket F_1(x) \rrbracket) + \Pi_{MulBA}(\llbracket z_2 \rrbracket^B, \llbracket x \rrbracket)$  and get secret share  $\llbracket y \rrbracket_A, \llbracket y \rrbracket_B$ 
12 return  $\llbracket y \rrbracket_A, \llbracket y \rrbracket_B$ 
13 End

```

The Softmax is defined as Eq. (5). Since the input to the exponential function fall within the range $(-\infty, 0]$ in Softmax, we approximation targets this domain. We decompose the input x into its integer part x_{int} and fractional part x_{frac} :

$$x = x_{int} + x_{frac} \quad (18)$$

and thus

$$\exp(x) = \exp(x_{int}) \cdot \exp(x_{frac}) \quad (19)$$

For the integer part, we adopt a lookup table to store the values of $\exp(x)$, where $x = \{-1, -2, \dots, -n\}$. Since $\exp(x)$ approaches zero as x gets closer to $-\infty$, only a small number of values are necessary to ensure precision. We set $n = 10$ based on empirical testing, which results in a compact LUT.

For the fractional part over $(-1, 0]$, a cubic polynomial can achieve low approximation error. We adopt Chebyshev method following our GELU approximation scheme. The resulting polynomial approximation $G(x)$ is as follows:

$$G(x) = 0.10267786x^3 + 0.46364971x^2 + 0.99307243x + 0.99978968 \quad (20)$$

We also apply Horner's rule to further reduce the number of multiplications. The exponential function computation algorithm is presented in Algorithm 4.

Algorithm 4: Exponential Function Computation

Input: input $x < 0$, threshold n , lookup table T , approximation polynomial $G(x)$

Output: approximation value y

```

1 Function Exp_Approx():
2   if  $x < -n$  then
3     // Check threshold
4     return 0
5   end
6    $x_{int} = \lceil x \rceil$  // Split out the integer part
7    $x_{frac} = x - x_{int}$  // Split out the fractional part
8    $A = T(x_{int})$  // Compute with a lookup table
9    $B = G(x_{frac})$  // Compute with the approximating polynomial
10   $y = A \cdot B$ 
11 return  $y$ 
12 End

```

In addition to exponentiation, Softmax also requires division. We compute the reciprocal using a reciprocal protocol Π_{rec} based on Goldschmidt's algorithm, as adopted in systems such as SIRNN [24]. This

transformation allows all operations to be expressed as multiplications, which are more efficient in secure computation frameworks.

5.3. Other non-linear function evaluation protocol

LayerNorm is another critical non-linear function in Transformer inference. We design a private LayerNorm protocol based on reciprocal square root optimization using Newton's iteration for its secure computation. The standard LayerNorm is given by Eq. (6). So:

$$\begin{aligned} \text{LayerNorm} &= \gamma \cdot \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta \\ &= \gamma \cdot \frac{n(x_i - \mu)}{n\sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2 + \epsilon}} + \beta \\ &= \sqrt{n} \cdot \gamma \cdot \frac{nx_i - n\mu}{\sqrt{\sum_{i=1}^n (nx_i - n\mu)^2 + \epsilon}} + \beta \end{aligned} \quad (21)$$

Let $y_i = nx_i - n\mu = nx_i - \sum_{i=1}^n x_i$, we get:

$$\text{LayerNorm} = \sqrt{n} \cdot \gamma \cdot \frac{y_i}{\sqrt{\sum_{i=1}^n y_i^2 + \epsilon}} + \beta \quad (22)$$

Therefore, there is a reciprocal square root operation, which is costly to compute directly in a privacy-preserving setting. We approximate $1/\sqrt{x}$ by Newton's Iteration method. Let

$$f(x) = \frac{1}{x^2} - a = 0 \quad (23)$$

so $x = 1/\sqrt{a}$. According to Newton's Iteration formula:

$$x_{n+1} = x_n - f(x_n)/f'(x_n) \quad (24)$$

we can get the iteration function:

$$x_{n+1} = 0.5x_n/(3 - ax_n^2) \quad (25)$$

Following the approach used in Falcon [47], we choose $2^{-[a/2]}$ as initial guess, and $2^a \leq x \leq 2^{a+1}$. We can then compute the reciprocal square root by Π_{rsqrt} based on Newton's Iteration. The details are provided in Algorithm 5.

Algorithm 5: LayerNorm Computation

Input: input vector $x < 0$, vector size n , parameters β and γ , a small constant ϵ

Output: value LayerNorm_i

```

1 Function Ln_Approx():
2    $sum = \sum_{i=1}^n x_i$  // Compute the sum of  $x_i$ 
3   for  $i = 1$  to  $n$  do
4      $y_i = nx_i - sum$  // Compute  $y_i$ 
5      $z_i = \Pi_{square} y_i$  // Using square protocol
6      $a = \sum_{i=1}^n z_i + \epsilon$ 
7      $t = \Pi_{rsqrt} a$  // Compute with Newton's Iteration
8      $\text{LayerNorm}_i = \sqrt{n} \cdot \gamma \cdot y_i \cdot t + \beta$  // Compute the values of layernorm
9   end
10  return  $\text{LayerNorm}_i$ 
11 End

```

6. Evaluations

In this section, we evaluate the performance of our privacy-preserving transformer inference scheme in terms of runtime, communication overhead and inference accuracy.

6.1. Experimental setup

We implement our scheme in C++, using the EzPC library¹ for secure MPC, and SEAL library for HE.² All experiments are conducted

Table 2

Evaluation of matrix multiplication protocol.

Method		Time(s)				Comm.(MB)
		LAN	WAN ₁	WAN ₂	WAN ₃	
Ct-Pt MatMul	IRON	41.03	442.14	233.15	209.25	4844.14
	BOLT	18.21	18.87	18.51	18.35	7.06
	NEXUS	200.62	200.62	200.62	200.62	/
	Ours	2.19	17.01	9.53	8.93	385.58
Ct-Ct MatMul	IRON	29.12	449.52	234.13	209.58	4918.38
	BOLT	10.56	13.91	13.17	11.92	9.88
	NEXUS	26.49	26.49	26.49	26.49	/
	Ours	0.92	5.63	3.41	3.16	129.03

on a machine with an Intel Core i7-10700K CPU (3.80 GHz, 8 cores, 16 threads) and 128 GB of RAM. We set the number of threads to 16. All results are averaged over 10 runs.

We use the CKKS with polynomial degree $N' = 16384$, which produces $N = N'/2 = 8192$ SIMD slots.

To simulate communication conditions, we control bandwidth using the Linux Traffic Control (tc) command. We evaluate performance under four network settings: LAN{3Gbps bandwidth, 0.8 ms latency}, WAN₁{200Mbps bandwidth, 40 ms latency}, WAN₂{400Mbps bandwidth, 40 ms latency}, WAN₃{400Mbps bandwidth, 20 ms latency}.

We compare our approach against existing works including Iron [33], BOLT [34] and NEXUS [35]. Specifically, Iron [33] and BOLT [34] adopt a similar MPC-HE framework based on EzPC and the BFV library, while NEXUS [35] serves as a state-of-the-art reference for fully HE-based end-to-end inference. Notably, we disable machine learning optimizations such that BOLT is evaluated without word elimination and NEXUS is tested without GPU acceleration. We conduct two types of benchmarks: microbenchmarks and end-to-end benchmarks in the following subsections. To evaluate the accuracy of our scheme, we use three datasets from the GLUE benchmark [48], which is a standard suite for evaluating LLM performance. It includes RTE, SST-2 and QNLI.

6.2. Microbenchmarks

We begin by evaluating the performance of individual building blocks, including matrix multiplication and non-linear function evaluation. Specifically, for matrix multiplication, we measure the runtime and communication overhead for both Ct-Pt MatMul and Ct-Ct MatMul. For non-linear function evaluation, we assess the runtime and communication overhead of the Private GELU, Softmax, and LayerNorm. Furthermore, we evaluate the accuracy of the polynomials used to approximate GELU and $\exp(x)$ in our scheme.

Matrix Multiplication. We compare our proposed Ct-Pt and Ct-Ct matrix multiplication schemes against baseline methods. The results are summarized in Table 2. Since NEXUS is a non-interactive scheme requiring only a one-time communication at initialization, its performance is invariant across different network settings, and it incurs zero communication overhead. For the Ct-Pt multiplication experiment, we use matrix dimensions of $128 \times 768 \times 64$. Our method achieves a runtime of 2.19 s in LAN, which is 18.7× faster than IRON, 8.3× faster than BOLT, and 91.6× faster than NEXUS. The communication overhead is 385.58 MB, which is 12.6× lower than IRON, but higher than BOLT. In the Ct-Ct multiplication experiment, we use matrix dimensions of $128 \times 128 \times 64$. Our scheme achieves a runtime of 0.92 s in LAN, which is 31.6× faster than IRON, 11.5× faster than BOLT and 28.8× faster than NEXUS. The communication overhead is 129.03 MB, which is 38.1× lower than IRON, though again slightly higher than BOLT. These results demonstrate that our matrix multiplication protocols provide substantial computational advantages, while there remains room for improvement in communication efficiency under constrained network environments.

¹ <https://github.com/mpc-msri/EzPC/tree/master/SCI>

² <https://github.com/microsoft/SEAL>

Table 3
Evaluation of non-linear function evaluation protocol.

Method		Time(s)				Comm.(MB)
		LAN	WAN ₁	WAN ₂	WAN ₃	
GELU	IRON	22.74	337.21	185.32	166.38	7525.14
	BOLT	8.55	129.11	76.85	63.41	2770.47
	NEXUS	56.13	56.13	56.13	56.13	/
	Ours	5.94	42.71	29.71	23.12	784.95
Softmax	IRON	10.93	155.42	92.52	76.73	3357.06
	BOLT	4.27	67.85	46.02	34.13	1285.09
	NEXUS	17.38	17.38	17.38	17.38	/
	Ours	1.07	23.12	22.86	12.13	24.34
LayerNorm	IRON	6.62	82.33	67.52	40.22	786.44
	BOLT	2.16	34.11	27.12	17.23	419.45
	NEXUS	12.35	12.35	12.35	12.35	/
	Ours	0.65	8.21	8.13	5.19	6.14

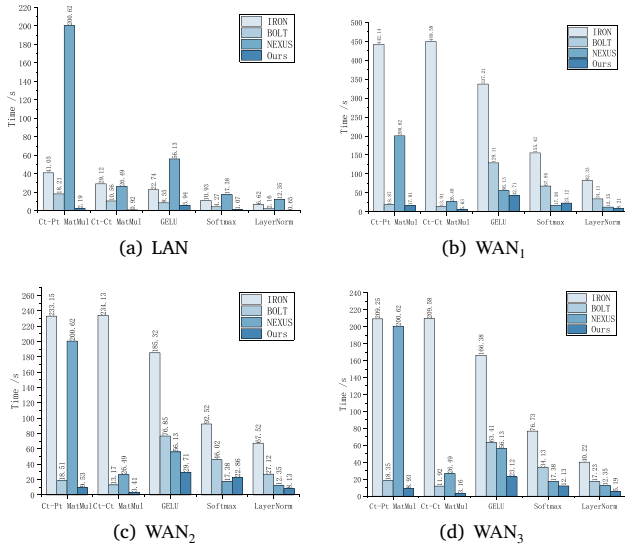


Fig. 5. Runtime of linear & non-linear operators.

Non-linear Function Evaluation. We also evaluate the runtime and communication cost of our GELU, Softmax, and LayerNorm protocols. Results are shown in Table 3. We set an input size of 128×3072 for GELU, our protocol achieves a runtime of 5.94 s in LAN, which is 3.8× faster than IRON, 1.4× faster than BOLT, and 9.4× faster than NEXUS. The communication overhead is 784.95 MB, 9.6× lower than IRON and 3.5× lower than BOLT. For Softmax, we set the input dimensions to 128×128 . Our method achieves a runtime of 1.07 s in LAN, which is 10.2× faster than IRON, 4.0× faster than BOLT, and 16.2× faster than NEXUS. The communication overhead is 24.34 MB, 137.9× lower than IRON, and 52.8× lower than BOLT. We set the LayerNorm's input dimensions to 128×768 . The runtime of our approach is 0.65 s in LAN, which is 10.2× faster than IRON, 3.3× faster than BOLT, and 19× faster than NEXUS. The communication overhead is 6.14 MB, 128.1× lower than IRON and 68.3× lower than BOLT.

Fig. 5(a)–(d) illustrate the runtime of all operators under different network conditions (LAN, WAN₁, WAN₂, WAN₃), while Fig. 6 compares the communication cost across these methods.

Accuracy evaluation of Polynomial Approximation. Since our private GELU and Softmax protocols use low-degree polynomial approximations, we also evaluate the approximation accuracy of these polynomials and compare them with those used in other schemes. Fig. 7(a) presents the function curves of our approximation, the true

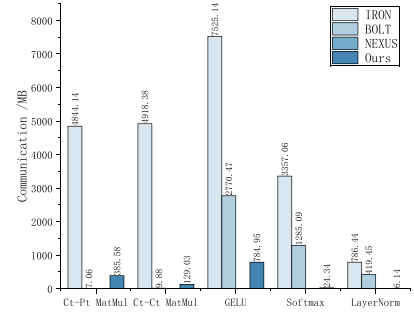


Fig. 6. Communication overhead of linear & non-linear operators.

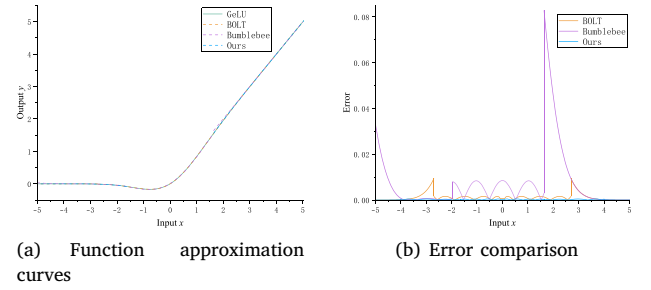


Fig. 7. GELU function comparisons.

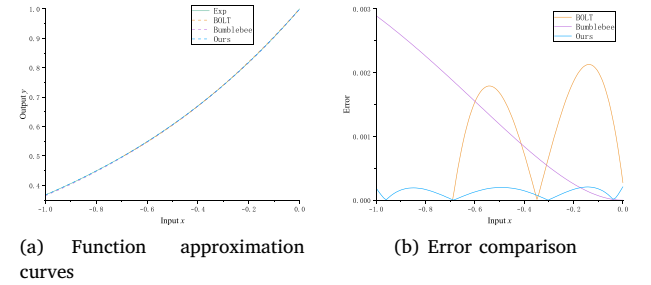


Fig. 8. Exponential function comparisons.

GELU function, and approximations from BOLT [34] and Bumblebee [31]. Fig. 7(b) shows the corresponding absolute error between each approximation and the true function. Our piecewise cubic polynomial achieves a maximum absolute error of 0.000466 and a mean absolute error of 0.000216. We then evaluate the polynomials used to approximate the exponential function in Softmax, as illustrated in Fig. 8. Our polynomial approximation, which is also a cubic polynomial, demonstrates significantly improved accuracy compared to the schemes used in BOLT and Bumblebee. Specifically, our method achieves a maximum absolute error of only 0.000210 and a mean absolute error of 0.000126. Table 4 presents the comparisons of the maximum and mean absolute errors for the polynomial approximations of the GELU and exponential functions. The error range for GELU is evaluated over $(-5, 5)$, and that for the exponential function is evaluated over $(-1, 0)$. As shown, our method achieves substantially lower errors than the baseline schemes in both cases.

Table 4Errors of GELU approximation in $(-5, 5)$ and exponential function approximation in $(-1, 0)$.

Method		Max absolute error	Mean absolute error
GELU Approximation	BOLT	9.35×10^{-3}	1.18×10^{-3}
	Bumblebee	3.33×10^{-2}	4.06×10^{-3}
	Ours	4.66×10^{-4}	2.16×10^{-4}
Exponential Function Approximation	BOLT	1.28×10^{-3}	2.13×10^{-3}
	Bumblebee	1.54×10^{-3}	4.25×10^{-3}
	Ours	2.10×10^{-4}	1.26×10^{-4}

Table 5

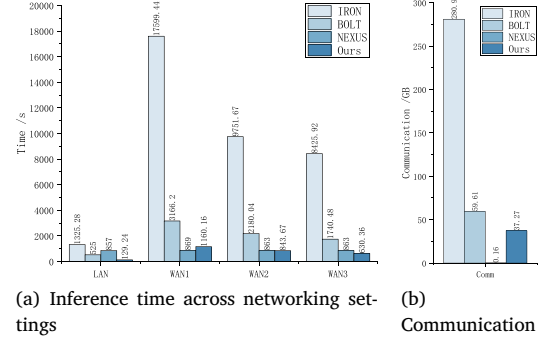
Hyperparameters of BERT-base.

Type	Value
Parameters number	110M
Transformer blocks	12
Hidden size	768
Attention heads	12

Table 6

Accuracy of BERT-base on the GLUE benchmarks.

Model	Dataset	Plaintext	Ours
BERT-base	RTE	70.04%	69.91%
	SST-2	92.36%	92.22%
	QNLI	90.30%	90.07%

**Fig. 9.** End-to-end comparisons using BERT-base.**Table 7**

End-to-end comparisons with the existing private inference frameworks (BERT-base).

Method	Time(s)				Comm.(GB)
	LAN	WAN ₁	WAN ₂	WAN ₃	
IRON	1325.28	17599.44	9751.67	8425.92	280.99
BOLT	525.0	3166.20	2180.04	1740.48	59.61
NEXUS	857	869	863	863	0.16
Ours	129.24	1160.16	843.67	630.36	37.27

6.3. End-to-End benchmarks

We now evaluate the end-to-end performance of our privacy-preserving Transformer inference scheme using the BERT-base model and compare it against existing frameworks such as IRON [33], BOLT [34], and NEXUS [35]. The hyperparameters of the BERT-base model are summarized in Table 5.

We first assess the impact of our scheme on inference accuracy. Only the GELU, Softmax, and LayerNorm functions are replaced with their privacy-preserving counterparts, while all other components of the model remain unchanged. We evaluate inference accuracy on three datasets from the GLUE benchmark: RTE, SST-2, and QNLI. Results are shown in Table 6. The accuracy of our scheme closely matches that of plaintext inference.

Next, we compare the total inference time and communication overhead of running BERT-base end-to-end under different network settings. Similar to NEXUS [35], the end-to-end performance is roughly the aggregation of the microbenchmarks. The results are presented in Table 7 and visualized in Fig. 9. In a LAN setting, our method achieves an inference time of 129.24 s, which is 10.2× faster than IRON, 4.1× faster than BOLT, and 6.6× faster than NEXUS. In terms of communication overhead, our method transmits 37.27 GB, which is 7.5× lower than IRON and 1.6× lower than BOLT. Due to its non-interactive design, NEXUS achieves the lowest communication overhead and is less affected by bandwidth and latency variations, which allows it to outperform our scheme in low-bandwidth or high-latency environments.

7. Related work

We categorize the existing work into two main areas: matrix multiplication and non-linear function evaluation.

7.1. Homomorphic matrix multiplication

For matrix multiplication, existing schemes primarily adopt two mainstream approaches: secret sharing (SS)-based and homomorphic encryption (HE)-based methods.

SS-based MatMul. MPCFormer [49] introduces a third-party dealer and uses Beaver triples for secure matrix multiplication, but it does not incorporate further optimizations to accelerate the operation. PUMA [37] builds its MatMul protocol on 2-out-of-3 replicated secret sharing. Each party performs local computation, then exchanges intermediate results through re-sharing, and finally combines them into a shared product. Other schemes such as [50,51] also follow similar approaches, but they typically incur high communication costs and frequent interaction rounds. Besides, they also require a third-party setup, which renders them less suitable for privacy-preserving inference in AIoT context, where decentralization and lightweight communication are critical.

HE-based MatMul. IRON [33], BOLT [34] and NEXUS [35] adopt a SIMD-style HE encoding method for parallel matrix multiplication. Specifically, IRON [33] encodes the plaintext matrices into polynomials and leverages HE to arrange coefficients in a way that aligns with the desired output format. BOLT [34] packs multiple plaintext data into a single ciphertext for SIMD parallelism and uses outer product representation with partial rotations to accelerate Ct-Ct MatMul. NEXUS [35] compresses several ciphertexts into one to minimize transmission overhead. Besides, BumbleBee [31] introduces an InterLeave packing scheme that efficiently combines multiple polynomials into one ciphertext, thereby reducing both ciphertext size and communication overhead. However, these HE-based methods still face heavy computational burdens due to the inherent complexity of ciphertext operations, and large-scale matrix multiplications in Transformer models further increase this overhead.

7.2. Non-linear function evaluation

Non-linear function evaluation in private Transformer inference primarily focuses on three functions: GELU, Softmax, and LayerNorm. Each approach for evaluating non-linear functions has its own trade-offs in terms of efficiency, accuracy, and computational complexity.

GELU. MPCFormer [49] uses a quadratic function $0.125x^2 + 0.25x + 0.5$ to approximate GELU. However, the approximation error becomes too large, making it necessary to introduce additional distillation to ensure inference accuracy. PUMA [37], BumbleBee [31], Bolt [34] and NEXUS [35] adopt low-degree polynomial approximations. Following a similar piecewise polynomial idea, they design their own approximation functions by selecting different segmentation points, segment counts, and polynomial degrees for each case. In addition, SIGMA [52] uses $\text{ReLU}(x) - \delta(x)$, where $\delta(x)$ is computed through a LUT. SHAFT [53] approximates GELU by the Fourier series.

Softmax. MPCFormer [49] replaces $\exp(x)$ in Softmax with cryptofriendly functions $(x + c)^2$, which leads to accuracy degradation. BOLT [34] decomposes the input into a non-negative integer and a float part, then applies a polynomial approximation. PUMA [37], BumbleBee [31], NEXUS [35], and CipherGPT [38] approximate $\exp(x)$ using the Taylor series, while IRON [33] uses look-up tables for exponential and reciprocal operations.

LayerNorm. NEXUS [35] introduces the *QuickSum* function to compute summations and uses Newton's iteration method to approximate the inverse square root. SIGMA [52] and CipherGPT [38] rely on LUTs for evaluating the reciprocal square root.

8. Conclusion

In this paper, we present an efficient privacy-preserving Transformer inference scheme for cloud-based intelligent decision-making in AIoT. We first develop a matrix multiplication protocol that enables fast ciphertext–plaintext and ciphertext–ciphertext operations within Transformer blocks. It leverages an offline-online collaborative computation pipeline, where only multiplication and accumulation operations of the corresponding SIMD ciphertexts are needed to achieve highly efficient matrix computation. Then we design a non-linear function evaluation protocol that supports accurate and efficient computation of GELU, Softmax, and LayerNorm under privacy constraints.

Compared to existing methods, experimental results demonstrate that our solution is $8.3\times$ – $91.6\times$ faster in matrix multiplication, $1.4\times$ – $19\times$ faster in non-linear function evaluation, reduces communication overhead by $3.5\times$ – $137.9\times$ in the LAN network, and achieves lossless accuracy. While the end-to-end benchmark results show that even for the BERT-base model, the latency and communication cost of a single private inference remain considerably high. When scaling to larger Transformer-based models, these costs would further increase to a level that may undermine the feasibility of real-world deployment. Therefore, to enhance the practicality of privacy-preserving inference, we will focus our future work on key techniques that further reduce communication overhead and accelerate privacy-preserving inference.

CRedit authorship contribution statement

Mingshun Luo: Writing – original draft, Software, Methodology, Investigation. **Haolei He:** Software, Methodology. **Wenti Yang:** Writing – review & editing. **Shuai Yuan:** Writing – review & editing. **Zhitao Guan:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (#62372173).

References

- [1] O.B. Sezer, E. Dogdu, A.M. Ozbayoglu, Context-aware computing, learning, and big data in internet of things: a survey, *IEEE Internet Things J.* 5 (1) (2017) 1–27.
- [2] M.A. Fadhel, A.M. Duham, A. Saihood, A. Sewify, M.N. Al-Hamadani, A. Albahri, L. Alzubaidi, A. Gupta, S. Mirjalili, Y. Gu, Comprehensive systematic review of information fusion methods in smart cities and urban environments, *Inf. Fusion* (2024) 102317.
- [3] R. Kumar, N. Agrawal, Analysis of multi-dimensional industrial IoT (IIoT) data in edge-fog-cloud based architectural frameworks: A survey on current state and research challenges, *J. Ind. Inf. Integr.* 35 (2023) 100504.
- [4] S.I. Siam, H. Ahn, L. Liu, S. Alam, H. Shen, Z. Cao, N. Shroff, B. Krishnamachari, M. Srivastava, M. Zhang, Artificial intelligence of things: A survey, *ACM Trans. Sens. Networks* 21 (1) (2025) 1–75.
- [5] A. Sreedevi, T.N. Harshitha, V. Sugumaran, P. Shankar, Application of cognitive computing in healthcare, cybersecurity, big data and IoT: A literature review, *Inf. Process. Manage.* 59 (2) (2022) 102888.
- [6] Gartner, Forecast: IoT market opportunity by technology segment, 2022–2028, 2024, Accessed: 2025-03, <https://www.gartner.com/document/5499495>.
- [7] A. Matin, M.R. Islam, X. Wang, H. Huo, G. Xu, AIoT for sustainable manufacturing: Overview, challenges, and opportunities, *Internet Things* 24 (2023) 100901.
- [8] Y. Hu, D. Ye, J. Kang, M. Wu, R. Yu, A cloud-edge collaborative architecture for multimodal LLMs-based advanced driver assistance systems in IoT networks, *IEEE Internet Things J.* (2024).
- [9] M.S. Murshed, C. Murphy, D. Hou, N. Khan, G. Ananthanarayanan, F. Hussain, Machine learning at the network edge: A survey, *ACM Comput. Surv.* 54 (8) (2021) 1–37.
- [10] C. Sivakumar, V. Mone, R. Abdumukhtor, Addressing privacy concerns with wearable health monitoring technology, *Wiley Interdiscip. Rev.: Data Min. Knowl. Discov.* 14 (3) (2024) e1535.
- [11] R.U. Rasool, H.F. Ahmad, W. Rafique, A. Qayyum, J. Qadir, Security and privacy of internet of medical things: A contemporary review in the age of surveillance, botnets, and adversarial ML, *J. Netw. Comput. Appl.* 201 (2022) 103332.
- [12] F.U.M. Ullah, K. Muhammad, I.U. Haq, N. Khan, A.A. Heidari, S.W. Baik, V.H.C. de Albuquerque, AI-assisted edge vision for violence detection in IoT-based industrial surveillance networks, *IEEE Trans. Ind. Informatics* 18 (8) (2021) 5359–5370.
- [13] H.R. Chi, C.K. Wu, N.F. Huang, K.F. Tsang, A. Radwan, A survey of network automation for industrial internet-of-things toward industry 5.0, *IEEE Trans. Ind. Informatics* 19 (2) (2022) 2065–2077.
- [14] S. Mobsite, N. Alaoui, M. Boulmalf, M. Ghogho, A privacy-preserving aIoT framework for fall detection and classification using hierarchical learning with multi-level feature fusion, *IEEE Internet Things J.* (2024).
- [15] Q. Liu, Y. Peng, Q. Xu, H. Jiang, J. Wu, T. Wang, T. Peng, G. Wang, S. Zhang, MARS: Enabling verifiable range-aggregate queries in multi-source environments, *IEEE Trans. Dependable Secur. Comput.* 21 (4) (2023) 1994–2011.
- [16] N. Wang, S. Wang, M. Li, L. Wu, Z. Zhang, Z. Guan, L. Zhu, Balancing differential privacy and utility: A relevance-based adaptive private fine-tuning framework for language models, *IEEE Trans. Inf. Forensics Secur.* (2024).
- [17] B. Xiao, B. Kantarci, J. Kang, D. Niyato, M. Guizani, Efficient prompting for llm-based generative internet of things, *IEEE Internet Things J.* (2024).
- [18] S. Zhang, Y. Pan, Q. Liu, Z. Yan, K.-K.R. Choo, G. Wang, Backdoor attacks and defenses targeting multi-domain ai models: A comprehensive review, *ACM Comput. Surv.* 57 (4) (2024) 1–35.
- [19] S. Wang, Q. Liu, Y. Xu, H. Jiang, J. Wu, T. Wang, T. Peng, G. Wang, Protecting inference privacy with accuracy improvement in mobile-cloud deep learning, *IEEE Trans. Mob. Comput.* 23 (6) (2023) 6522–6537.
- [20] Z.Á. Mann, C. Weinert, D. Chabal, J.W. Bos, Towards practical secure neural network inference: the journey so far and the road ahead, *ACM Comput. Surv.* 56 (5) (2023) 1–37.
- [21] D. Rathee, M. Rathee, N. Kumar, N. Chandran, D. Gupta, A. Rastogi, R. Sharma, Cryptflow2: Practical 2-party secure inference, in: *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020, pp. 325–342.
- [22] Z. Huang, W.j. Lu, C. Hong, J. Ding, Cheetah: Lean and fast secure {Two-Party} deep neural network inference, in: *31st USENIX Security Symposium*, USENIX Security 22, 2022, pp. 809–826.
- [23] J. Feng, Y. Wu, H. Sun, S. Zhang, D. Liu, Panther: Practical secure 2-party neural network inference, *IEEE Trans. Inf. Forensics Secur.* (2025).
- [24] D. Rathee, M. Rathee, R.K.K. Goli, D. Gupta, R. Sharma, N. Chandran, A. Rastogi, Simn: A math library for secure rnn inference, in: *2021 IEEE Symposium on Security and Privacy*, SP, IEEE, 2021, pp. 1003–1020.

- [25] A. Diao, L. Fenaux, T. Humphries, M. Dietz, F. Ebrahimiaghazani, B. Kacsmar, X. Li, N. Lukas, R.A. Mahdavi, S. Oya, et al., Fast and private inference of deep neural networks by co-designing activation functions, in: 33rd USENIX Security Symposium, USENIX Security 24, 2024, pp. 2191–2208.
- [26] W. Ao, V.N. Boddeti, {AutoFHE}: Automated adaption of {CNNs} for efficient evaluation over {FHE}, in: 33rd USENIX Security Symposium, USENIX Security 24, 2024, pp. 2173–2190.
- [27] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, *Adv. Neural Inf. Process. Syst.* 30 (2017).
- [28] W. Yang, X. Li, M. Li, Z. Zhang, Z. Guan, L. Zhu, VSecNN: Verifiable and privacy-preserving neural network inference in cloud service, *IEEE Trans. Inf. Forensics Secur.* 20 (2025) 10541–10555.
- [29] W. Yang, Z. Guan, L. Wu, Z. He, A secure neural network inference framework for intelligent connected vehicles, *IEEE Netw.* 38 (6) (2024) 120–127.
- [30] Z. He, W. Yang, L. Wu, Z. Guan, SecureBadger: A homomorphic encryption-based framework for secure medical inference, *Digit. Commun. Networks* (2025).
- [31] W.j. Lu, Z. Huang, Z. Gu, J. Li, J. Liu, C. Hong, K. Ren, T. Wei, W. Chen, Bumblebee: Secure two-party inference framework for large transformers, 2023, *Cryptology ePrint Archive*.
- [32] C. Zeng, D. He, Q. Feng, X. Yang, Q. Luo, SecureGPT: A framework for multi-party privacy-preserving transformer inference in GPT, *IEEE Trans. Inf. Forensics Secur.* (2024).
- [33] M. Hao, H. Li, H. Chen, P. Xing, G. Xu, T. Zhang, Iron: Private inference on transformers, *Adv. Neural Inf. Process. Syst.* 35 (2022) 15718–15731.
- [34] Q. Pang, J. Zhu, H. Möllering, W. Zheng, T. Schneider, Bolt: Privacy-preserving, accurate and efficient inference for transformers, in: 2024 IEEE Symposium on Security and Privacy, SP, IEEE, 2024, pp. 4753–4771.
- [35] J. Zhang, X. Yang, L. He, K. Chen, W.-j. Lu, Y. Wang, X. Hou, J. Liu, K. Ren, X. Yang, Secure transformer inference made non-interactive, 2024, *Cryptology ePrint Archive*.
- [36] N.P. Smart, F. Vercauteren, Fully homomorphic SIMD operations, *Des. Codes Cryptogr.* 71 (2014) 57–81.
- [37] Y. Dong, W.-j. Lu, Y. Zheng, H. Wu, D. Zhao, J. Tan, Z. Huang, C. Hong, T. Wei, W. Chen, Puma: Secure inference of llama-7b in five minutes, 2023, *arXiv preprint arXiv:2307.12533*.
- [38] X. Hou, J. Liu, J. Li, Y. Li, W.-j. Lu, C. Hong, K. Ren, Ciphergpt: Secure two-party gpt inference, 2023, *Cryptology ePrint Archive*.
- [39] J. Devlin, M.W. Chang, K. Lee, K. Toutanova, Bert: Pre-training of deep bidirectional transformers for language understanding, in: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 2019, pp. 4171–4186.
- [40] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, et al., Language models are unsupervised multitask learners, *OpenAI Blog* 1 (8) (2019) 9.
- [41] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al., Llama: Open and efficient foundation language models, 2023, *arXiv preprint arXiv:2302.13971*.
- [42] J. Yuan, W. Liu, J. Shi, Q. Li, Approximate homomorphic encryption based privacy-preserving machine learning: a survey, *Artif. Intell. Rev.* 58 (3) (2025) 82.
- [43] J.H. Cheon, A. Kim, M. Kim, Y. Song, Homomorphic encryption for arithmetic of approximate numbers, in: *Advances in Cryptology—ASIACRYPT 2017: 23rd International Conference on the Theory and Applications of Cryptology and Information Security*, Hong Kong, China, December 3–7, 2017, *Proceedings, Part I* 23, Springer, 2017, pp. 409–437.
- [44] H. Qin, D. He, Q. Feng, M.K. Khan, M. Luo, K.K.R. Choo, Cryptographic primitives in privacy-preserving machine learning: A survey, *IEEE Trans. Knowl. Data Eng.* 36 (5) (2023) 1919–1934.
- [45] B. Knott, S. Venkataraman, A. Hannun, S. Sengupta, M. Ibrahim, L. van der Maaten, Crypten: Secure multi-party computation meets machine learning, *Adv. Neural Inf. Process. Syst.* 34 (2021) 4961–4973.
- [46] Y. Lindell, How to simulate it—a tutorial on the simulation proof technique, *Tutorials Cryptogr.: Dedic. To Oded Goldreich* (2017) 277–346.
- [47] S. Wagih, S. Tople, F. Benhamouda, E. Kushilevitz, P. Mittal, T. Rabin, Falcon: Honest-majority maliciously secure framework for private deep learning, 2020, *arXiv preprint arXiv:2004.02229*.
- [48] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, S.R. Bowman, GLUE: A multi-task benchmark and analysis platform for natural language understanding, 2018, *arXiv preprint arXiv:1804.07461*.
- [49] D. Li, R. Shao, H. Wang, H. Guo, E.P. Xing, H. Zhang, Mpcformer: fast, performant and private transformer inference with mpc, 2022, *arXiv preprint arXiv:2211.01452*.
- [50] Y. Akimoto, K. Fukuchi, Y. Akimoto, J. Sakuma, Privformer: Privacy-preserving transformer with mpc, in: 2023 IEEE 8th European Symposium on Security and Privacy, EuroS&P, IEEE, 2023, pp. 392–410.

- [51] Y. Wang, G.E. Suh, W. Xiong, B. Lefaudeaux, B. Knott, M. Annaram, H.H.S. Lee, Characterization of mpc-based private inference for transformer-based models, in: 2022 IEEE International Symposium on Performance Analysis of Systems and Software, ISPASS, IEEE, 2022, pp. 187–197.
- [52] K. Gupta, N. Jawalkar, A. Mukherjee, N. Chandran, D. Gupta, A. Panwar, R. Sharma, Sigma: Secure gpt inference with function secret sharing, 2023, *Cryptology ePrint Archive*.
- [53] A.Y. Kei, S.S. Chow, SHAFT: Secure, handy, accurate, and fast transformer inference, in: *Network and Distributed System Security Symposium*, Vol. 2025, NDSS, 2025.



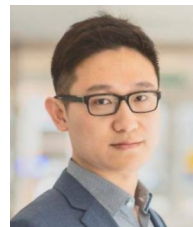
Mingshun Luo received the M.S. degree in cyberspace security from North China University of Technology, Beijing, China, in 2024. He is currently working toward the Ph.D. degree with North China Electric Power University, Beijing. His research interests include privacy-preserving machine learning and privacy computing.



Haolei He received the B.E. degree from North China Electric Power University, Beijing, China, in 2024. He is now studying artificial intelligence with North China Electric Power University, Beijing. His research interests include privacy-preserving machine learning and privacy computing.



Wenti Yang received the Ph.D. degree in Information Security from the North China Electric Power University, Beijing, China, in 2024. She is currently an Assistant Researcher with the Department of New Networks, Pengcheng Laboratory (PCL), Shenzhen, Guangdong, China. Her research focuses on privacy-preserving machine learning and security in computing power networks.



Shuai Yuan received the Ph.D. degree in management science and systems from State University of New York at Buffalo, Buffalo, NY, USA. He is an Assistant Professor with the Department of Finance, Operations, and Information Systems with Brock University, St. Catharines, ON, Canada. His research interests include business analytics, resource management in the cloud, and high-performance computing.



Zhitao Guan received the B.E and Ph.D. degrees in computer application from Beijing Institute of Technology, Beijing, China, in 2002 and 2008, respectively. He is currently a Professor with the School of Control and Computer Engineering, North China Electric Power University, Beijing. His research interests include blockchain, AI security, and privacy computing. He has authored more than 100 peer-reviewed journal and conference papers in these areas.